

JavaScript DOM

Manipulando documentos HTML com JavaScript

Selecionar

Buscar elementos na árvore do documento

Manipular

Alterar conteúdo, estilos e atributos

Criar

Inserir e remover nós dinamicamente

Navegar

Percorrer a hierarquia da árvore DOM

O que é o DOM?

O DOM (Document Object Model) é uma interface de programação web. O navegador analisa o documento HTML e cria uma representação em memória chamada de **árvore (Tree)**. Cada ramificação dessa árvore termina em um **nó (Node)**, que contém objetos representando elementos HTML, atributos ou até mesmo texto.

Conceitos Fundamentais

document

Objeto global que representa a página HTML dentro do navegador

Tree

Estrutura em árvore criada pelo navegador ao analisar o HTML

Node

Cada nó da árvore representa um elemento, atributo ou texto

Inspecionando o documento

Para visualizar a representação completa da árvore do documento no console do navegador, utilize:

```
// Exibe a representação da árvore
// do documento no console
console.dir(document);
```

- ❗ O método `console.dir()` exibe uma listagem interativa das propriedades do objeto `document`, permitindo explorar toda a estrutura do DOM.

Buscar Elementos

Os métodos de busca são a porta de entrada para qualquer manipulação do DOM. Escolha o método adequado conforme o tipo de seletor e a necessidade de performance.

1

querySelector

Retorna o **primeiro elemento** que corresponda ao seletor CSS especificado.

```
const card =  
document.querySelector('.card');
```

2

querySelectorAll

Retorna uma **NodeList estática** com todos os elementos que correspondam ao seletor CSS.

```
const botoes =  
document.querySelectorAll('button'  
);
```

3

getElementById

Retorna a referência do elemento pelo seu atributo `id` – **extremamente rápido**.

```
const cabecalho =  
document.getElementById('main-  
header');
```

Buscar Elementos por Classe e Tag

Além dos seletores CSS universais, o DOM oferece métodos especializados que retornam **HTMLCollections em tempo real** — atualizadas automaticamente quando o documento muda.



getElementsByClassName

Retorna uma **HTMLCollection em tempo real** de todos os elementos com a classe especificada.

```
const itensLista =  
document.getElementsByClassName('list-item');
```



getElementsByTagName

Retorna uma **HTMLCollection** de elementos com o nome da tag HTML informada.

```
const links =  
document.getElementsByTagName('a');
```



getElementsByTagNameNS

Retorna elementos com uma tag específica pertencente a um **URI de namespace** (XML ou SVG).

```
const svgs =  
document.getElementsByTagNameNS('http://www.w3.org/2000/svg', 'path');
```

📌 **HTMLCollection vs NodeList:** HTMLCollections são *vivas* (refletem mudanças no DOM em tempo real), enquanto NodeLists retornadas por `querySelectorAll` são *estáticas*.

Acessar Conteúdo

Três propriedades essenciais para ler e modificar o conteúdo de elementos HTML. A escolha correta entre elas impacta tanto a segurança quanto a performance da aplicação.

textContent

Retorna ou define o conteúdo de texto de um nó — **ignora tags HTML**. Mais seguro contra XSS.

```
titulo.textContent = 'Novo  
Título da Página';
```

innerHTML

Obtém ou define a marcação HTML interna do elemento — **renderiza tags HTML**. Use com cautela.

```
container.innerHTML =  
'<strong>Aviso  
importante!</strong>';
```

value — Elementos de Formulário

Lê ou define o valor de elementos de formulário como `<input>`, `<select>` e `<textarea>`.

```
console.log(inputNome.value); // Lê o valor  
inputNome.value = "";        // Limpa o campo
```

textContent

Texto puro, sem HTML

innerHTML

Markup HTML completo

value

Formulários e inputs

Alterar Estilos

O DOM oferece duas abordagens para manipular a aparência visual dos elementos: estilos inline via `style` e gerenciamento de classes via `classList`. A segunda é a prática recomendada.

`style` — Estilos Inline

Acessa e modifica diretamente o estilo inline de um elemento. Propriedades com hífen são convertidas para **camelCase**.

```
caixa.style.backgroundColor = 'blue';
caixa.style.marginTop = '20px';
```

⚠ Estilos inline têm alta especificidade e dificultam a manutenção. Prefira `classList` sempre que possível.

`classList` — Gerenciamento de Classes

Retorna a coleção de classes do elemento — **melhor forma** de manipular estilos via CSS.

```
menu.classList.add('ativo'); // Adiciona classe
menu.classList.remove('oculto'); // Remove classe
menu.classList.toggle('dark'); // Adiciona ou remove
```

add()

Adiciona uma ou mais classes

remove()

Remove uma classe existente

toggle()

Alterna presença da classe

Criar e Inserir Elementos

A criação dinâmica de elementos é um dos recursos mais poderosos do DOM. O processo segue sempre dois passos: **criar o elemento na memória** e depois **inserir-lo na árvore**.



Criar Elemento

Inserir no DOM

createElement

Cria o elemento HTML especificado na memória — **precisa ser inserido no DOM depois**.

```
const novoParagrafo = document.createElement('p');  
novoParagrafo.textContent = 'Parágrafo gerado via JS.';
```

appendChild

Adiciona um nó ao **final da lista de filhos** de um nó pai.

```
const divArea = document.getElementById('area');  
divArea.appendChild(novoParagrafo);
```

Ler e Alterar Atributos

Os atributos HTML (como `src`, `href`, `disabled`, `readonly`) podem ser lidos, definidos, verificados e removidos programaticamente através de quatro métodos complementares.

1

getAttribute

Retorna o **valor** do atributo especificado.

```
const caminho = imagem.getAttribute('src');
```

2

setAttribute

Define ou **atualiza** o valor de um atributo.

```
botao.setAttribute('disabled', 'true');
```

3

hasAttribute

Retorna `true` ou `false` indicando se o elemento **possui** o atributo.

```
if (link.hasAttribute('target')) {  
  console.log('O link abre em nova aba.');
```

4

removeAttribute

Remove o atributo especificado de um elemento.

```
input.removeAttribute('readonly');
```

Remover Elementos

O DOM oferece dois métodos para remoção de elementos. O método moderno `remove()` é mais simples, enquanto `removeChild()` é o método clássico que exige referência ao elemento pai.

`remove()` — Método Moderno

Método moderno que remove o **próprio elemento** da árvore do DOM. Simples e direto.

```
const modal = document.querySelector('.modal-flutuante');
modal.remove(); // O elemento deleta a si mesmo
```

Vantagem: não é necessário conhecer o elemento pai — o próprio elemento se remove.

`removeChild()` — Método Clássico

Remove um nó filho do DOM — deve ser **chamado a partir do elemento pai**.

```
const lista = document.getElementById('minha-lista');
const primeiroItem = lista.querySelector('li');
lista.removeChild(primeiroItem);
```

Uso: necessário quando se tem referência ao pai e ao filho que se deseja remover. Compatível com navegadores mais antigos.

📘 Em projetos modernos, prefira `element.remove()` pela sua sintaxe mais limpa. Use `removeChild()` apenas quando precisar de compatibilidade com navegadores legados.

Inserção Avançada

Além do `appendChild`, o DOM oferece métodos mais precisos para controlar **exatamente onde** um novo elemento será inserido em relação aos elementos existentes.

1

`prepend`

Inserir nós no **início** do elemento pai, antes do seu primeiro filho.

```
lista.prepend(novoItem);
```

2

`insertBefore`

Inserir um nó **antes do nó de referência** como filho de um nó pai.

```
pai.insertBefore(novoElemento, elementoReferencia);
```

3

`insertAdjacentElement`

Inserir um elemento em posição relativa: `beforebegin`, `afterbegin`, `beforeend`, `afterend`.

```
titulo.insertAdjacentElement('afterend', subtitulo);
```

4

`insertAdjacentHTML`

Semelhante ao anterior, mas analisa e insere uma **string como HTML**.

```
caixa.insertAdjacentHTML('beforebegin', '<p>Texto antes</p>');
```

Navegando pela Árvore do DOM

Uma vez que você tem referência a um elemento, é possível **navegar pela árvore** para acessar seus parentes, filhos e irmãos sem precisar fazer novas buscas no documento.



parentElement / parentNode

Retorna o **nó pai** do elemento atual.

```
const listaPai = item.parentElement;  
// Retorna <ul> ou <ol>
```



children

Retorna uma **HTMLCollection viva** com apenas os elementos filhos (ignora texto e comentários).

```
const todasImagens = galeria.children;  
console.log(`A galeria tem  
${todasImagens.length} imagens.`);
```



nextElementSibling / previousElementSibling

Retornam o elemento imediatamente **posterior ou anterior** no mesmo nível hierárquico.

```
const proximoSlide =  
ativo.nextElementSibling;  
const slideAnterior =  
ativo.previousElementSibling;
```

- ❏ A navegação pela árvore é especialmente útil em componentes dinâmicos como carrosséis, listas ordenáveis e menus — onde a relação entre elementos é mais importante do que seus identificadores.